

Iterative Methods

- Golub & van Loan: Matrix Computation

- HW 1 (end at wk 3), HW 2 (end at wk 6) ← 25%

- 1 month: implement/reproduce paper → presentation at the end

Goal: either $Ax = b$ or $Ax = \lambda x$ for sparse & large A .
← 50% at grade
← $\alpha(n)$ nonzero elements

Topics:

- classical iterative methods (Gauss-Jacobi, Gauss-Seidel, SOR)

- Krylov methods (conjugate gradient, Generalized minimum residual method, MINRES, CG)

- Preconditioning (most practical to optimize convergence rates)

- eigen solvers

Applications

- Solving PDEs numerically

- Inverse Problems (topology optimization, PDE-constrained optimization)

- Machine Learning

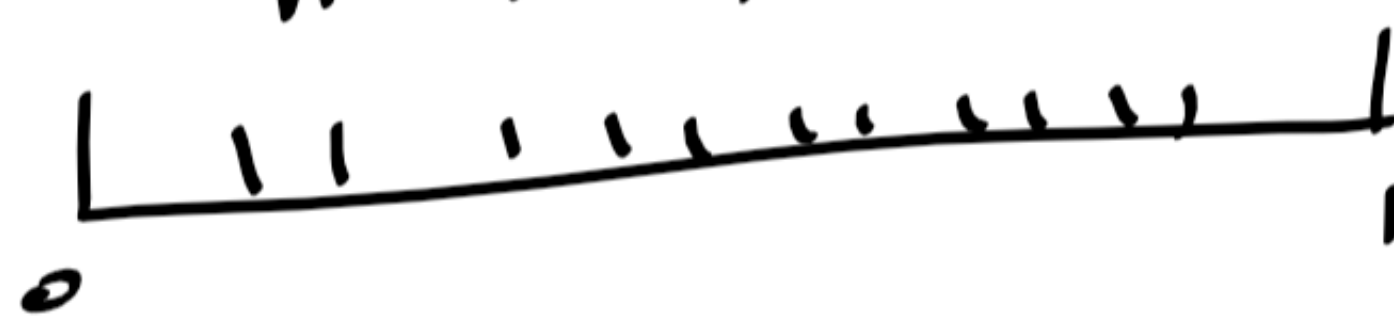
Poisson 1D:

$$-\frac{\partial^2 u}{\partial x^2} = f(x), \quad x \in (0, 1)$$

$$u(0) = u(1) = 0$$

finite differences: $\frac{\partial^2 u}{\partial x^2} \approx \frac{u(x+h) - 2u(x) + u(x-h))}{h^2} + O(h^2)$ (derive from Taylor Series)

$n+2$ points, $h = \frac{1}{n+1}$



$$x_i = ih \text{ for } i \in [0, n+1] \rightarrow \frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} \approx f_i$$
$$u_i = u(x_i), \quad f_i = f(x_i)$$

$$\rightarrow -u_{i-1} + 2u_i - u_{i+1} = h^2 f_i \rightarrow$$

$$\underbrace{\begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & -1 & \\ & & \ddots & \ddots & \ddots \\ & & & -1 & 2 \end{bmatrix}}_{L_T} \cdot \underbrace{\begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_n \\ u_{n+1} \end{bmatrix}}_x = h^2 \underbrace{\begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix}}_b$$

$$Au = f$$

$$1D: A = T$$

$$2D: \Omega = [0,1] \times [0,1]$$

$$\begin{cases} -\Delta u = f, & x \in \Omega \\ u = 0, & x \in \partial\Omega \end{cases}$$

$$A_{2D} = T \otimes I + I \otimes T$$

for B $m \times n$, C $p \times q$, define

$$B \otimes C = \begin{bmatrix} b_{11}C & b_{12}C & \dots & b_{1n}C \\ b_{21}C & & & \vdots \\ \vdots & & & \\ b_{m1}C & \dots & b_{mn}C \end{bmatrix} \in \mathbb{R}^{mp \times nq}$$

$$3D: A_{3D} = T \otimes I \otimes I + I \otimes T \otimes I + I \otimes I \otimes T$$

→ really clean structure, so its easy to analyze; this will be our benchmark.

In 3D, matrix size is $O(10^4) - O(10^9)$

Inverse Problem

$$\min_{m, u} \frac{1}{2} \|u(m) - d\|_2^2 + R(m) \quad \text{s.t.} \quad A(m)u = f \quad \begin{cases} \text{for fluids, divergence free PDE} \\ \nabla \cdot (\eta(x) \nabla u) = f(x) \end{cases}$$

→ m, u knows state

→ solving a large PDE many times

Direct Methods

$Ax = b \xrightarrow{\text{direct}}$ solve w/ QR, LU, banded solvers

iterative $\hookrightarrow$ relaxation methods, krylov methods, multigrid methods

$\downarrow$
workhorse for practical applications, others for preconditioning

direct methods find solutions in a predictable number of steps

— LU: factor $A = LU$ or $PA = LU$

$\left\{ \begin{array}{l} \text{solve } Ly = b, \text{ then } u = y \\ \text{in general } O(n^3) \text{ Flops to generate, } O(n^2) \text{ to solve.} \end{array} \right.$ $\swarrow$ $O(n^2)$ sparse complexity

— If A is banded w/ upper/lower bandwidth p, q

$\rightarrow$ LU preserves bandwidth $\rightarrow O(npq)$ Flops to generate, $O(n(p+q))$ to solve/store

QR: construct orthogonal Q & upper triangular R w/ $A = QR$

$\rightarrow Ax = b \Rightarrow QRx = b \Rightarrow Rx = Q^T b$ and $x = R^{-1}(Q^T b)$

$\swarrow$ A upper Hessenberg $\downarrow$ $O(n^2)$ generation instead of $O(n^3)$

In practice, can build $Q^T b$ and back sub for $x \rightarrow$ no need to solve.

General: $O(n^3)$ generation, $O(n^2)$ solving/storage

Remarks

— Direct solvers $\swarrow$ all or nothing don't have intermediate results — iterative solvers do.

— You can't get fast approximations by lowering the tolerance

— $O(n^3)$ is just crazy

Iterative methods
even for singular A , some iterative solvers can find a solution $\leftarrow$ ^{for} underdetermined systems

Procedure:

$$- x^0 \rightarrow x^1 \rightarrow x^2 \rightarrow \dots \rightarrow x^k \quad w/ \quad \lim_{k \rightarrow \infty} x^k = x^* \quad w/ \quad Ax^* = b$$

- easier to parallelize than direct.

Classic (stationary) iterative methods

↑ update rule does not vary between steps

1. Gauss-Jacobi
2. Weighted GS
3. Gauss-Seidel
4. SOR

$$A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ a_{21} & \dots & a_{2n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

1. Given x^k , cycle through each equation and set x_i^{k+1} treating other x_j as constant in row:

$$a_{i1} x_1^k + \dots + a_{i,i-1} x_{i-1}^k + \underline{a_{i,i} x_i^{k+1}} + a_{i,i+1} x_{i+1}^k + \dots + a_{in} x_n^k = b_i$$

$$\rightarrow x_i^{k+1} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^k}{a_{ii}} \rightarrow \text{diagonal needs to be nonzero.}$$

each row's computation is independent $\rightarrow$ parallelize!

$\rightarrow$ GS is the most friendly for CPUs.

- block version: update a block of 3 rows at once to get a system of equations (used for things like solving for 3d coordinates).

$\rightarrow$ don't normally do this.

2. Weighted GS

(i) Calculate $\tilde{x}^{k+1}$ w/ GS

(ii) Set $x^{k+1} = \omega \tilde{x}^{k+1} + (1-\omega)x^k$

$\omega \in (0,1)$, relaxation parameter $\rightarrow$ dampening

Use it when GS is too oscillatory $\rightarrow$ want dampening.

$\rightarrow$ Common for hyperbolic PDEs.

$\omega \in (1,2) \rightarrow$ faster convergence

$\omega \geq 2 \rightarrow$ divergence

double check
intervals

3. Gauss-Seidel

- difficult to parallelize

- Same idea as GS, but you use the latest information.

$$a_{i1}x_1^{k+1} + a_{i2}x_2^{k+1} + \dots + a_{i,i-1}x_{i-1}^{k+1} + \underbrace{a_{ii}x_i^{k+1}}_{\text{impossible to parallelize}} + a_{i,i+1}x_{i+1}^k + \dots + a_{in}x_n^k = b_i$$

$$\rightarrow x_i^{k+1} = \frac{b_i - \sum_{j < i} a_{ij}x_j^{k+1} - \sum_{j > i} a_{ij}x_j^k}{a_{ii}}$$

impossible to parallelize.

- You can still parallelize: colored GS is where you do parallel sweeps per color if the DOFs are in a colored graph (block matrices) ask?

- Changing the order at the sweep changes the solutions and even the order of convergence $\leftarrow$ actively being researched ("Randomized Gauss-Seidel").

- Sometimes works better for large scale ML projects.

- Similar # of FLOPs (relatively inexpensive)

- Sequential / parallelizable

- GS usually has faster convergence

- SoR:

(i) $\tilde{x}^{k+1}$ from GS

(ii) $x^{k+1} = \omega \tilde{x}^{k+1} + (1-\omega)x^k$

Stopping Condition

tol ϵ .

① $\frac{\|Ax^k - b\|}{\|b\|} < \epsilon_1$ ② $\frac{\|x^{k+1} - x^k\|}{\|x^{k+1}\|} < \epsilon_2$

↑ check relative residual ↑ still making practice

Normally, we don't have an absolute tolerance; relative tolerance is safer. If you have physical context (e.g. scale, units), absolute error works.

→ usually watch both.

Convergence (order / criteria)

Review:

- Norm, $v \in \mathbb{R}^n$, $\|v\|_1 = \sum_{i=1}^n |v_i|$, $\|v\|_\infty = \max_i |v_i|$, $\|v\|_2 = \sqrt{\sum_i |v_i|^2}$

- All norms on $\mathbb{R}^n$ are equivalent: given 2 norms $\|\cdot\|_p$ and $\|\cdot\|_q$,

$\exists c_1, c_2$ s.t. $c_1 \|u\|_p \leq \|u\|_q \leq c_2 \|u\|_p \quad \forall u \in \mathbb{R}^n$.

→ we can use whatever norm we want.

- For $1 \leq p \leq n$, $\|v\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}$.

- For ML algorithms, p -norm can be used as a differentiable alternative to $\|\cdot\|_\infty$ (which is not differentiable). ← for autograd.

- $\|x\|_\infty \leq \|x\|_1 \leq \sqrt{n} \|x\|_2$

$\|x\|_\infty \leq \|x\|_2 \leq \sqrt{n} \|x\|_\infty$

$\|x\|_\infty \leq \|x\|_1 \leq n \|x\|_\infty$

- Spectral radius $\rho(A) = \max_{\lambda \in \mathbb{C}} |\lambda|$.

- Matrix norm: $\|A\|_p = \sup_{\|x\|_p=1} \frac{\|Ax\|_p}{\|x\|_p} = \sup_{\|x\|_p=1} \|Ax\|_p$

← worst case scale factor $\Leftrightarrow \rho(A)$

$\|A\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|$, $\|A\|_1 = \max_j \sum_{i=1}^n |a_{ij}|$, $\|A\|_2 = \sup_{\|x\|_2=1} \|Ax\|_2 = \sigma_1$

↑ max row sum

↑ max column sum

? → largest singular value

Review (matrix norm)

$$\|A\|_\infty = \max_i \sum_j |a_{ij}|, \quad \|A\|_1 = \max_j \sum_i |a_{ij}|, \quad \|A\|_2 = \sigma_{\max}, \quad \|A\|_F = \sqrt{\sum |a_{ij}|^2}$$

For $p \in \{1, 2, \infty, F\}$

$$\|A x\|_p = \|A\|_p \|x\|_p$$

$$\|AB\|_p = \|A\|_p \|B\|_p$$

$$\|A\|_2 \leq \|A\|_p \leq \sqrt{n} \|A\|_2$$

$$\|A\|_2 \leq \sqrt{\|A\|_1 \|A\|_\infty}$$

$$\rho(A) = \max_i |\lambda_i|$$

we care about this

— we will use norms as a proxy for $\rho(A)$

Since $\rho(A) \leq \|A\|_p$ (easier to bound $\|A\|_p$).

$$1. A^k \rightarrow 0 \text{ if } \rho(A) < 1.$$

$$2. \text{ If } A = A^* \text{ (symmetric matrices), the } \|A\|_2 = \rho(A)$$

$$3. \text{ If } A \text{ normal } (A^* A = A A^*) \Leftrightarrow A = U \Lambda U^* \quad \text{orthonormal eigenbasis}$$

$U^* U = I$

Splitting Iteration

Let $A = M - N$, M nonsingular. Then

fixed point problem

$$Ax = b \Leftrightarrow Mx = b + Nx \Leftrightarrow x = M^{-1}(b + Nx)$$

FPI:

$$x^{k+1} = M^{-1}(Nx^k + b) = \underbrace{M^{-1}N}_{\text{This is being applied again and again}} x^k + M^{-1}b$$

Let $G = M^{-1}N$. Let x^* be true solution:

$$\begin{cases} Mx^* = b + Nx^* \\ Mx^{k+1} = b + Nx^k \end{cases}$$

$$M(x^* - x^{k+1}) = N(x^* - x^k)$$

$$M e^{k+1} = N e^k$$

$$\rightarrow e^{k+1} = M^{-1}N e^k = G e^k = G^{k+1} e^0$$

We want $\|G^k e^0\| \xrightarrow{k \rightarrow \infty} 0 \quad \forall \text{ initial guess and initial } e^0$.

→ equivalent to saying $G^k \rightarrow 0$ or $\rho(G) < 1$.

Thm: Let $A = M - N$, M nonsingular, then $x^{k+1} = M^{-1}(Nx^k + b)$ converges
 $\forall$ initial x^0 if $\rho(M^{-1}N) < 1$.

→ Convergence speed is related to ρ (smaller ρ better).

→ $\|G\|_p < 1$, then $\|G^k e^0\|_p \rightarrow 0$, recall $\rho(G) \leq \|G\|_p$ for any norm.
 Note, $\|G\|_p > 1$ is not enough for div. ($\rho < 1 < \|G\|_p$ still possible).

Let: $A = L + D + U$ (strict lower, diag, strict upper)

GS: $M = D, N = -(L+U)$

→ $Dx = b - (L+U)x$ assume nonzero diagonal.
 → $x^{k+1} = D^{-1}(b - (L+U)x^k)$

GS (ω): $A = L + D + U + \frac{1}{\omega}D - \frac{1}{\omega}D$

$$x^{k+1} = \omega D^{-1}b - \omega D^{-1} \left[L + \left(1 - \frac{1}{\omega}\right)D + U \right] x^k$$

$$M = \frac{1}{\omega}D, \quad N = - \left[\left(1 - \frac{1}{\omega}\right)D + L + U \right]$$

GS: $(L+D)x^{k+1} = b - Ux^k$

$$x^{k+1} = (L+D)^{-1}(b - Ux^k)$$

$$M = L+D, \quad N = -U$$

→ need forward substitution for $M^{-1}b$ → sequential

Sol: $(L + \frac{1}{\omega}D)x = b - (U + D - \frac{1}{\omega}D)x$

$$x^{k+1} = (L + \frac{1}{\omega}D)^{-1} \left[b - (U + D - \frac{1}{\omega}D)x^k \right]$$

$$M = L + \frac{1}{\omega}D, \quad N = - \left[U + (1 - \omega)D \right]$$

⑤ SSOR (sets rid at bias for it $L = U^T$)

→ does forward/backward sweep in some iteration.

$$\underbrace{\left(\frac{1}{\omega} D + L\right)}_{L_N} y^{k+1} = \underbrace{\left((\frac{1}{\omega} - 1) D - L^T\right)}_{L_N} x^k + b$$

$$\underbrace{\left(\frac{1}{\omega} D + L^T\right)}_{L_N} x^{k+1} = \underbrace{\left((\frac{1}{\omega} - 1) D - L\right)}_{L_N} y^{k+1} + b$$

You can show: $M = \frac{\omega}{2-\omega} \left(\frac{1}{\omega} D + L\right) D^{-1} \left(\frac{1}{\omega} D + L^T\right)$

↑ work this out?

$$N = \frac{\omega}{2-\omega} \left((\frac{1}{\omega} - 1) D - L\right) D^{-1} \left((\frac{1}{\omega} - 1) D - L^T\right)$$

Strictly Diagonally dominant: $\sum_{j \neq i} |a_{ij}| < |a_{ii}| \forall i$.

Thm: if A is strictly diagonal dominant, GS converges $\forall x_0$.

→ HW

- usually, more dominant $\Rightarrow$ faster convergence
- weighted GS has the same sufficient condition (proof?)

If GS already converges, then any $0 < \omega < 1$ (under-relaxed) will also converge.

Pf: long, but read through it.

If A is SPD (symmetric positive definite), GS converges.

Pf: $C_{gs} = -(D+L)^{-1} U = -(D+L)^{-1} L^T \leftarrow$ by symmetry.

Similarity transpose & preserves λ):

$$C_1 := D^{1/2} C_{gs} D^{-1/2} = D^{1/2} (D+L)^{-1} L^T D^{-1/2}$$

$$\text{let } \tilde{L} = D^{-1/2} L D^{-1/2} \rightarrow D+L = D^{1/2} (I + \tilde{L}) D^{1/2}$$

$$(D+L)^{-1} = D^{-1/2} (I + \tilde{L})^{-1} D^{-1/2}$$

Cont

$$\rightarrow G = - \underbrace{D^{1/2} D^{-1/2}}_{I} (I + \tilde{L})^{-1} \underbrace{D^{-1/2} L^T D^{-1/2}}_{\tilde{L}^T}$$

$$= - (I + \tilde{L})^{-1} \tilde{L}^T$$

$\rightarrow$ In effect, we are "absorbing" D into L to make $\tilde{L}$.

Now consider the eigenvalues of G .

$$G v = \lambda v, \quad v^H v = 1 \quad (\text{normalized eigenvectors}).$$

$\downarrow$

$$\rightarrow v^H = v^T, \quad \|v\|_2 = 1.$$

$$-(I + \tilde{L})^{-1} \tilde{L}^T v = \lambda v$$

$$-\tilde{L}^T v = \lambda (I + \tilde{L}) v$$

$$-v^H \tilde{L}^T v = \lambda (v^H v + v^H \tilde{L} v)$$

$$-v^H \tilde{L}^T v = \lambda (1 + v^H \tilde{L} v) \quad a+bi$$

$$\tilde{L}^T = \tilde{L}^H \quad \text{since } L \text{ is real}$$

$$|\lambda|^2 = \left| \frac{a+bi}{1+a+bi} \right|^2 = \frac{a^2+b^2}{1+2a+a^2+b^2} \leq \frac{a^2+b^2+2a+1}{a^2+b^2+2a+1} = 1$$

$$\begin{aligned} |\lambda|^2 &\leq 1 \\ \rightarrow |\lambda| &\leq 1 \end{aligned}$$

Since A is SPD:

$$D^{-1/2} A D^{1/2} = I + \tilde{L} + \tilde{L}^T$$

is also SPD

$\downarrow$

$$1 + v^H \tilde{L} v + v^H \tilde{L}^T v \geq 0$$

$$1 + (a+bi) + (a-bi) \geq 0$$

$$1 + 2a \geq 0$$

Convex matrix:

$$v^H \tilde{L}^H v = v^H \tilde{L} v = a-bi$$

$\rightarrow$ all eigenvalues lie in unit circle, so $\rho(A) = 0$.

- SPD $\neq$ diagonal dominant, so GS is not sufficient conditions for GS to converge (but we want parallelization).

- You can use ω to decrease $\rho(A)$.

SOR

If A is symmetric, then SOR converges if A SPD & $\omega < 2$.

Look it up

- Think about how to choose ω in practice
 - You can think of it like an optimization problem.
-

Remarks

- we usually want to bound the spectral radius of the iteration matrix, $\rho(M^{-1}N)$
- we can do this using norms if we only want to prove it is sufficient

Thm: Suppose A is consistently ordered. (treat A as a graph, edge $i \leftrightarrow j$ if $a_{ij} \neq 0$. Consistently ordered means you can assign rows so each graph only connects to neighboring layers; example: banded matrices like tridiagonal matrices)

- $M_{\omega}^{-1} N_{\omega}$ only has real eigenvalues and $\mu := \rho(M_{\omega}^{-1} N_{\omega})$.
- we can switch to SOR w/ optimal $\omega = \frac{2}{1 + \sqrt{1 - \mu^2}}$, $\rho(M_{\text{SOR}}^{-1} N_{\text{SOR}}) = \omega_{\text{opt}} - 1$.
- GS(ω) converges for $0 < \omega < 1$
- SSOR converges for A SPD & $\omega < 2$.

Power Method

- Motivation, we care about $\rho(A)$ as well as the eigenvalues.
- know $\rho(A)$ lets us compare methods.

Goal: $\rho(A) = \max_j |\lambda_j|$, $A = \mathbb{C}^{n \times n}$, A diagonalizable.

Eigen decomposition $\rightarrow$ $V^{-1} A V = \Lambda = \begin{bmatrix} \lambda_1 & & \\ & \lambda_2 & \\ & & \ddots \end{bmatrix}$, $V = [v_1, \dots, v_n]$.
 $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|$

Algorithm: Init $q^{(0)} \in \mathbb{C}^n$ s.t. $\|q^{(0)}\|_2 = 1$ (typically choose basis vector).

$$z^{(k)} = A q^{(k-1)} \rightarrow q^{(k)} = \frac{z^{(k)}}{\|z^{(k)}\|_2}, \quad \lambda^{(k)} = q^{(k)*} A q^{(k)}$$

Rayleigh Quotient: $R_A(q) = \frac{q^* A q}{q^* q}$, if $\|q\|_2 = 1$, $R_A(q) = q^* A q$.

If q is an eigenvector, then $\lambda^{(k)} = R_A(q^{(k)})$ estimates dominant eigenvalue $\leftarrow q^{(k)}$ is dominant eigenvector

Intuition: These power steps "boosts" components associated w/ v_1 (dominant eigenvector).

Assume $q^{(0)} \neq 0$ since v_i 's are linearly indep, $q^{(0)} = \alpha_1 v_1 + \dots + \alpha_n v_n$.

If A is normal, then $\{v_i\}$ is orthonormal $\rightarrow \alpha_1 = v_1^* q^{(0)}$. Further, assume

$$\langle q^{(0)}, v_1 \rangle \neq 0 \rightarrow \alpha_1 \neq 0.$$

If A is not normal, then $\{v_i\}$ is not orthogonal ($\Rightarrow \langle q^{(0)}, v_1 \rangle \not\rightarrow 0$). Instead,

assume, $q^{(0)} \notin \text{span}\{v_2, v_3, \dots, v_n\} \rightarrow \alpha_1 \neq 0$.

$$\rightarrow \alpha_1 \neq 0.$$

Then, $A^k q^{(0)} = \alpha_1 \lambda_1^k v_1 + \alpha_2 \lambda_2^k v_2 + \dots + \alpha_n \lambda_n^k v_n$
$$= \alpha_1 \lambda_1^k \left[v_1 + \frac{\alpha_2}{\alpha_1} \left(\frac{\lambda_2}{\lambda_1}\right)^k v_2 + \dots + \frac{\alpha_n}{\alpha_1} \left(\frac{\lambda_n}{\lambda_1}\right)^k v_n \right]$$

Since $q^{(k)} \in \text{span}\{A^k q^{(0)}\}$, wts. $\text{dist}(\text{span}\{q^{(k)}\}, \text{span}\{v_1\}) = O\left(\frac{\lambda_2}{\lambda_1}\right)^k$

$$\text{and } |\lambda_1 - \lambda^{(k)}| = O\left(\frac{\lambda_2}{\lambda_1}\right)^k$$

$$\text{ie. } A^k q^{(0)} = \alpha_1 \lambda_1^k \left[v_1 + O\left(\left(\frac{\lambda_2}{\lambda_1}\right)^k\right) \right] \rightarrow q^{(k)} \sim v_1.$$

Appendix: distance between subspaces.

- $\text{span} \{u_1, u_2, \dots, u_k\}$

- If $u \in \mathbb{R}^n$ and $\|u\|_2 = 1$, then uu^T is the projection onto $\text{span} \{u\}$

eg $w \in \mathbb{R}^n$, $(uu^T)w = u(u^T w) = u \langle u, w \rangle = u \|w\| \cos \theta$ (projection)

- If $U = [u_1, u_2, \dots, u_k]$, $U^T U = I$ and UU^T is the projection onto $\text{span} \{u_1, u_2, \dots, u_k\}$

If $\|u\|_2 = \|v\|_2 = 1$, $\text{dist}(\text{span} \{u\}, \text{span} \{v\}) = \frac{\text{norm of diff at proj. matrices.}}{\|uu^T - vv^T\|_2} = \sin \theta$.

Let $c = u^T v$, $s = \sqrt{1-c^2}$, let $w = \frac{v - cu}{s}$.

Clearly, $w^T u = \frac{v^T - cu^T}{s} u = \frac{v^T u - c}{s} = 0 \rightarrow w \perp u$

So $\{u, w\}$ is orthonormal: $v = cu + sw$. Let $u \sim \begin{bmatrix} 1 \\ 0 \end{bmatrix} \rightarrow uu^T = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$
 $v \sim \begin{bmatrix} c \\ s \end{bmatrix} \rightarrow vv^T = \begin{bmatrix} c^2 & cs \\ cs & s^2 \end{bmatrix}$

$$\rightarrow B = uu^T - vv^T = \begin{bmatrix} 1-c^2 & -cs \\ -cs & -s^2 \end{bmatrix} = \begin{bmatrix} s^2 & -cs \\ -cs & -s^2 \end{bmatrix}$$

$$\det(B - \lambda I) = \lambda^2 - \text{tr}(B)\lambda + \det(B) = 0$$

$$\hookrightarrow \lambda^2 - s^2 = 0 \rightarrow \lambda = \pm s$$

$$\rightarrow B \text{ is symmetric, so } \|B\|_2 = \rho(s) = \max\{|s|, |-s|\} = |s|$$

$$\|uu^T - vv^T\|_2 = s = \sqrt{1-c^2} = \sqrt{1-(u^T v)^2} = \sqrt{1-\cos^2 \theta} = |\sin \theta|$$

Back to problem:

$$\|q^{(k)} q^{(k)T} - v_1^T v_1^T\|_2 = \sin \theta, \quad \sin \theta \rightarrow 0 \text{ as } \left(\frac{\lambda_2}{\lambda_1}\right)^k \rightarrow 0, \text{ so } q^{(k)} \parallel v \text{ as } k \rightarrow \infty$$

Remark

- Cannot have $a_i = 0$ (if you do - choose a new value)

- we also need $|z_1| \geq |z_2| \geq \dots \geq |z_n|$ (not 2) $\leftarrow$ proof also works for nondiagonalizable matrices $\rightarrow$ use Jordan form.

Power helps estimate $\rho(A)$, so it can also be used to estimate optimal ω in S_{ω}^2

HW1

$$c_n = f \rightarrow A_n = f$$

$$A = \begin{bmatrix} 2^{-1} & & & \\ -1 & 2^{-1} & & \\ & -1 & 2^{-1} & \\ & & \ddots & \ddots \\ & & & -1 & 2^{-1} \\ & & & & -1 & 2^{-1} \end{bmatrix}$$

$$\begin{bmatrix} \sin(\frac{k\pi}{n+1}) \\ \sin(\frac{2k\pi}{n+1}) \\ \vdots \\ \sin(\frac{nk\pi}{n+1}) \end{bmatrix}$$

$$\sin(\frac{n\pi}{n+1}) \left[2 - 2\cos(\frac{n\pi}{n+1}) \right]$$

$$2\sin(\frac{k\pi}{n+1}) - 2\sin(\frac{n\pi}{n+1}) \cos(\frac{n\pi}{n+1})$$

$$\left[2\sin(\frac{k\pi}{n+1}) - \sin(\frac{2k\pi}{n+1}) \right]$$

$$-\sin(\frac{k\pi(j-1)}{n+1}) + 2\sin(\frac{k\pi j}{n+1}) - \sin(\frac{k\pi(j+1)}{n+1})$$

$$\left[\sin\left(\frac{k\pi j}{n+1} - \frac{k\pi}{n+1}\right) + \sin\left(\frac{k\pi j}{n+1} + \frac{k\pi}{n+1}\right) \right]$$

$$2\sin(\frac{k\pi j}{n+1}) - 2\sin(\frac{k\pi j}{n+1}) \cos(\frac{k\pi}{n+1})$$

$$= \sin(\frac{k\pi j}{n+1}) \cdot \left(2 - 2\cos(\frac{k\pi}{n+1}) \right)$$

$$-\sin(\frac{k\pi(n-1)}{n+1}) + 2\sin(\frac{n\pi k}{n+1})$$

Conjugate Gradient Method

- classical iteration schemes are slow but cheap.
 more than n iter for non.
 often need reduction parameter (might be expensive)
- especially bad if you are changing A in $Ax=b$ (to calculate ω).

Read Textbook derivation of CG

Background: A is SPD, or atleast semidefinite

Def: vectors $\{p_i\}$ are A -orthogonal if $\langle p_i, Ap_j \rangle = \begin{cases} 0 & i \neq j \\ \neq 0 & i = j \end{cases} \quad \forall i, j.$

$$\langle v, Aw \rangle = \langle v, w \rangle_A \quad \leftarrow \text{notation}$$

- $\langle v, Aw \rangle = \langle v, w \rangle_A$ defines an inner product (and a norm: $\langle v, Av \rangle = \|v\|_A$)
- A can't be singular (then what happens to CG) $\leftarrow ?$, $\langle v, w \rangle_A$ is well defined if A is SPD.

Lemma: If A is SPD and $\{p_i\}_{i=1}^n$ are A -orthogonal, the $\{p_i\}_{i=1}^n$ is linear indep. ($\Rightarrow$ form a basis)

Pf: Let $\{c_i\}$ s.t. $\sum_{i=1}^n c_i p_i = 0$.
 not all nonzero

Choose non zero c_i , then $\langle p_i, A \sum_{j=1}^n c_j p_j \rangle = 0$

For $i \neq j$, $\langle p_i, Ap_j \rangle = 0$

$$\rightarrow \langle p_i, A c_i p_i \rangle = c_i \|p_i\|_A^2$$

A is SPD $\rightarrow$ this can't be zero $\rightarrow$ contradiction.

Derive CG given magic $\{p_i\}$ $\leftarrow$ need to generate (Gram-Schmidt?)

$Ax=b$ (given A SPD), $\{p_i\}$ are A -orthogonal $\rightarrow$ want $x = \sum_{j=1}^n \alpha_j p_j$

$$\begin{aligned} \rightarrow A \left(\sum_{j=1}^n \alpha_j p_j \right) &= b \\ \sum_{j=1}^n \alpha_j (A p_j) &= b \end{aligned}$$

For any p_i , consider $\langle p_i, Ax \rangle = \langle p_i, b \rangle$
 $\langle p_i, \sum_{j=1}^n \alpha_j (A p_j) \rangle = \alpha_i \langle p_i, A p_i \rangle = \langle p_i, b \rangle$
 $\rightarrow \alpha_i = \frac{\langle p_i, b \rangle}{\langle p_i, A p_i \rangle}$
 regular euclidean inner product.
 very, very cheap (challenge is getting p_i)

$$Ax=b \Rightarrow x = \sum_{j=1}^n \alpha_j p_j, \quad \alpha_i = \frac{\langle p_i, b \rangle}{\langle p_i, A p_i \rangle}$$

What we care about is if I construct these A -orthogonal $\{p_i\}$, and compute $\{\alpha_i\}$, does $x = \sum_{j=1}^n \alpha_j p_j$, does $Ax=b$?

Pf: Bruc, assume $Ax-b \neq 0$. Because $\{p_i\}$ is a basis for $\mathbb{R}^n$, $\exists k$ s.t. $\langle p_k, Ax-b \rangle \neq 0$.

$$\rightarrow \langle p_k, A \sum_{i=1}^n \alpha_i p_i - b \rangle = \langle p_k, A \alpha_k p_k - b \rangle = \alpha_k \langle p_k, A p_k \rangle - \langle p_k, b \rangle \stackrel{\text{def of } \alpha_k}{=} 0$$

$\rightarrow$ Contradiction.

- Suppose we have a process to generate a sequence of orthogonal $\{p_i\}$.
 → this defines an iterative procedure for solving $Ax=b$ that will terminate in n steps since we have a basis for $\mathbb{R}^n$ (with the exception of floating point errors).

- As $\{p_i\}$ is generated:

$$x^{(1)} = \frac{\langle p_1, b \rangle}{\langle p_1, Ap_1 \rangle} p_1$$

$$x^{(2)} = x^{(1)} + \frac{\langle p_2, b \rangle}{\langle p_2, Ap_2 \rangle} p_2$$

$$x^{(k)} = x^{(k-1)} + \frac{\langle p_k, b \rangle}{\langle p_k, Ap_k \rangle} p_k$$

↑
accumulating
solution
components.

Terminating early gives you the projection of x onto $\text{span}(p_1, p_2, \dots, p_k)$.

→ logical basis for A-orthogonalization methods.

1. In exact arithmetic, $x^{(n)} = x^*$ ($Ax^* = b$)

→ in a way, it is a direct method.

- very practical if you want to solve 1M 12×12 linear systems in parallel (block matrices for PDEs). ← on GPU

→ lets you balance computations on each thread (hard code iteration count on each thread for 1).

2. All methods have $x^{(k)}$ ($k < n$) that may closely approximate x^*

Conventions:

- Let $x^{(0)} = 0$. Then $r = Ax - b \rightarrow r_0 = Ax^0 - b = Ax^0 - Ax^* = Ae^{(0)}$

→ $r = Ae^{(0)}$ is an equivalent problem → solve for e w/ $e^{(0)} = 0$.

→ $e = x^{(0)} - x^* \rightarrow x^* = x^{(0)} - e$ solves $Ax^* = b$.

Practical VI^* :

- actual Ch is more subtle *

Init $k=0$, $x^{(k)} = 0$, $r_0 = b$.

while $\frac{\|r_k\|}{\|b\|} > \epsilon$

$k = k+1$

determine p_k s.t. $p_k \perp_A p_i$ ($\forall i < k$).

$\alpha_k = \langle p_k, b \rangle / \langle p_k, p_k \rangle_A$

$x^{(k)} = x^{(k-1)} + \alpha_k p_k$

$r_k = b - Ax^{(k)}$

← expensive way to increment r_k ,
 Ch doesn't need matrix mul.

we can lose A-orthogonality from accumulated errors.

Next time!

$$A \text{ is SPD} \rightarrow \begin{cases} \langle v, w \rangle_A = \langle v, Aw \rangle \\ Ax = b \end{cases}$$

→ A-orthogonal vectors are linearly independent

$$\{p_j\} \text{ is A-orthogonal and } x = \sum_{j=1}^n \alpha_j p_j \text{ w/ } \alpha_j = \frac{\langle p_j, b \rangle}{\langle p_j, Ap_j \rangle}$$

The challenge is generating a good A-orthogonal basis (we generate these one at a time). $x^{(k)} = x^{(k-1)} + \alpha_k p_k$

$$\begin{aligned} k=0, & \quad x^{(0)} = \vec{0}, \quad r_0 = b \\ \text{while } \frac{\|r_k\|}{\|b\|} > \epsilon: & \\ \quad k++ & \\ \quad \text{Find } p_k \perp_A p_j \quad (\forall j < k) & \\ \quad \alpha_k = \langle b, p_k \rangle / \langle p_k, Ap_k \rangle & \\ \quad x^{(k)} = x^{(k-1)} + \alpha_k p_k & \\ \quad r_k = b - Ap_k & \end{aligned}$$

Fact 1:

$$r_k = r_{k-1} - \alpha_k \underbrace{Ap_k}_{\text{already computed this}}$$

Fact 2:

$$\langle b, p_k \rangle = \langle r_j, p_k \rangle \quad (\forall j < k)$$

$$\rightarrow \langle b, p_k \rangle = \langle r_{k-1}, p_k \rangle$$

Fact 3:

$$\langle r_k, p_j \rangle = 0 \quad (\forall j < k)$$

→ easy to Gram-Schmidt into something A-orthogonal

$$1/ \quad r_k = b - Ax^k = b - A \sum_{j=1}^k \alpha_j p_j = \underbrace{b - A \sum_{j=1}^{k-1} \alpha_j p_j}_{= r_{k-1}} - \alpha_k Ap_k$$

$$\rightarrow r_k = r_{k-1} - \alpha_k Ap_k$$

$$2/ \quad \forall j < k, \quad \langle \underbrace{Ap_j}_{\text{definition}}, p_k \rangle = 0 \text{ by definition.}$$

$$\langle p_j, p_k \rangle = \langle p_{j-1} - \alpha_j Ap_j, p_k \rangle = \langle r_{j-1}, p_k \rangle$$

$$\langle r_j, p_k \rangle = \langle r_{j-1}, p_k \rangle = \langle r_{j-2}, p_k \rangle = \dots = \langle r_0, p_k \rangle = \langle b, p_k \rangle$$

$$\rightarrow \langle r_j, p_k \rangle = \langle b, p_k \rangle$$

$$Ax=b \Rightarrow A \sum_{i=1}^n \alpha_i p_i$$

$$\begin{aligned} 3/ \quad \langle r_k, p_j \rangle &= \langle b - Ax^k, p_j \rangle = \langle b - A \sum_{i=1}^k \alpha_i p_i, p_j \rangle = \langle A \left[\sum_{i=1}^n \alpha_i p_i - \sum_{i=1}^k \alpha_i p_i \right], p_j \rangle \\ &= \langle A \sum_{i=k+1}^n \alpha_i p_i, p_j \rangle \stackrel{j < k, p_j \perp_A p_i}{=} 0 \end{aligned}$$

$$\rightarrow \langle r_k, p_j \rangle = 0$$

→ we optimize our algorithm

$$k=0, x^0=0, r_0=b.$$

while $\frac{\|r_k\|}{\|b\|} > \epsilon$:

$k++$

Determine $P_k \perp_A P_j \ (\forall j < k)$

$$\alpha_k = \frac{\langle r_{k-1}, P_k \rangle}{\langle P_k, AP_k \rangle}$$

$$x^{(k)} = x^{(k-1)} + \alpha_k P_k$$

$$r_k = r_{k-1} - \alpha_k AP_k$$

already
computed

Last piece before CG is the Gram-Schmidt to find P_k .

A: given random linearly independent vectors, use Gram-Schmidt

B: A-orthogonalize Krylov vectors $\{b, Ab, A^2b, \dots\}$
→ most optimal order to generate A-orthogonal vectors.

C: A-orthogonalize residuals (equivalent to Krylov vectors, only for CG. Most efficient for now).

We will consider (B) more formally later.

Inefficient since we don't want to store $\{b, Ab, A^2b, \dots\}$.

Recall for (c), we already have a linearly indep. vectors $\{P_1, P_2, \dots, P_{k-1}, r_{k-1}\}$
Note, we can also just have $r_k = 0 \rightarrow$ problem is solved, but this is a trivial case.

→ Run Gram-Schmidt on r_k to find P_k w/ $\langle \cdot, \cdot \rangle_A$

$$P_k = r_{k-1} - \frac{\langle r_{k-1}, AP_{k-1} \rangle}{\langle P_{k-1}, AP_{k-1} \rangle} P_{k-1} - \frac{\langle r_{k-1}, AP_{k-2} \rangle}{\langle P_{k-1}, AP_{k-2} \rangle} P_{k-2} + \dots = r_{k-1} - \langle r_{k-1}, AP_{k-1} \rangle \frac{P_{k-1}}{\|P_{k-1}\|_A^2} - \dots$$

Want: $\sum_{j=1}^{k-2} \frac{\langle r_{k-1}, AP_j \rangle}{\langle P_j, AP_j \rangle} P_j = 0 \rightarrow P_k = r_{k-1} - \frac{\langle r_{k-1}, AP_{k-1} \rangle}{\langle P_{k-1}, AP_{k-1} \rangle} P_{k-1}$

Fact 1: $\langle r_j, r_k \rangle = 0 \ \forall j < k$

Fact 2: If $P_k = r_{k-1} - \frac{\langle r_{k-1}, AP_{k-1} \rangle}{\langle P_{k-1}, AP_{k-1} \rangle} P_{k-1}$, then $\langle P_k, AP_j \rangle = 0 \ \forall j < k$
→ all lower terms in GS are 0.

Note: Krylov's subspaces (option B) is equivalent: $\text{span}\{P_1, P_2, \dots, P_k\} = \text{span}\{b, Ab, A^2b, \dots, A^kb\}$
→ Textbook's approach? we will prove when we talk about Krylov subspaces.

Next time: Convergence of CG.

Example Problems

$$A \in \mathbb{R}^{n \times n}, \quad x^{(k+1)} = M^{-1} N x^{(k)} + M^{-1} b$$

$$\text{Jacobi: } M = D, \quad N = -(L+U)$$

$$\text{Weighted Jacobi (SOR): } M = \frac{1}{\omega} D, \quad N = \left((1 - \frac{1}{\omega}) D + L + U \right)$$

$$G_J = D^{-1} (-L-U) = I - D^{-1} A$$

$$G_{SOR} = I - \omega D^{-1} A$$

$$\text{Jacobi converges} \rightarrow \rho(G_J) < 1 \rightarrow \forall (z, u) \text{ in } G_J, |z| < 1.$$

$$(I - D^{-1} A) u = \lambda u$$

$$D^{-1} A u = (1 - \lambda) u$$

$$\rightarrow (I - \omega D^{-1} A) u = (1 - \omega(1 - \lambda)) u$$

$$\rightarrow 1 - \omega(1 - \lambda) \text{ is an eigenvalue of } G_{SOR}$$

$$\text{Want } 1 - \omega(1 - \lambda) < 1, \quad \lambda = x + yi \rightarrow x^2 + y^2 \leq 1 \leftarrow \rho(G_J) < 1.$$

$$\rightarrow 1 - |1 - \omega(1 - \lambda)|^2 = 1 - |(1 - \omega) + \omega(x + yi)|^2$$

$$= \dots \stackrel{\text{algebra}}{=} \omega (2(1-x) - \omega((1-x)^2 + y^2))$$

If $0 < \omega \leq 1$, then we can show this is greater than 0 $\rightarrow \rho(G_{SOR}) < 1$
 $\rightarrow$ SOR converges.

Catch on up C G notes b b

weedy $\rightarrow$ Last time: $\|x^* - x^{(k)}\|_A \leq 2 \|x^* - x^{(0)}\|_A \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k$ $\kappa = \text{condition}$

$$\frac{\|x^* - x^{(k)}\|_A}{\|x^* - x^{(0)}\|_A} \leq \min_{\substack{\deg(q) \leq k \\ q(0)=1}} \max_j |q(\lambda_j)|$$

We will use Chebyshev polynomials: $\begin{cases} T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \\ T_0(x) = 1, T_1(x) = x \end{cases}$

Thm: $n \in \mathbb{N}, T_n = \inf_{\deg(q) \leq n-1} \left[\max_{[-1,1]} (x^n + Q(x)) \right]$

$\rightarrow \min T_n = \frac{1}{2^{n-1}}$ is achieved by $x^n + Q(x) = \frac{1}{2^{n-1}} T_n(x)$ find proof?

i.e. $\frac{1}{2^{n-1}} T_n(x)$ is the monic polynomial of degree n of least size over $[-1, 1]$

For CL , we need polynomials over $[z_{\min}, z_{\max}]$.

$\rightarrow g: [z_{\min}, z_{\max}] \rightarrow [-1, 1] \Rightarrow g(s) = -1 + 2 \frac{s - z_{\min}}{z_{\max} - z_{\min}}$ linear affine map

$\Rightarrow \hat{q}(s) = T_n \left(-1 + 2 \frac{s - z_{\min}}{z_{\max} - z_{\min}} \right)$ is the optimal Chebyshev.

We still need $q(0) = 1$, we need to normalize it by dividing by $\hat{q}(0)$.

$$q(s) = \left(T_k \left(-1 + 2 \frac{s - z_{\min}}{z_{\max} - z_{\min}} \right) \right) / T_k \left(-\frac{z_{\min} + z_{\max}}{z_{\max} - z_{\min}} \right)$$

Now, we want to calculate $\inf_{\deg(q) \leq n-1} \left(\max_j |q(\lambda_j)| \right)$.

Let $\alpha = z_{\min}, \beta = z_{\max}$

$\rightarrow q(s) = \frac{T_k \left(-1 + 2 \frac{s - \alpha}{\beta - \alpha} \right)}{T_k \left(-\frac{\alpha + \beta}{\beta - \alpha} \right)}$

Since $-1 + 2 \frac{s - \alpha}{\beta - \alpha} \in [-1, 1], T_k(\cdot) \leq 1$
since $T_k(x)$ is a Chebyshev Polynomial.

$\rightarrow \max_j |q(s)| = \left| T_k \left(-\frac{\alpha + \beta}{\beta - \alpha} \right) \right|^{-1}$ which doesn't have s in it anymore.

Since $R(A) := \frac{z_{\max}}{z_{\min}} = \frac{\beta}{\alpha} \rightarrow -\frac{\alpha + \beta}{\beta - \alpha} = \frac{1 + \beta/\alpha}{1 - \beta/\alpha} = \frac{1 + R}{1 - R}$

Since $T_k(-x) = T_k(x)$ for even k , $T_k(-x) = -T_k(x)$ for odd k , and we take the absolute value, we can remove the minus sign $\rightarrow \left| T_k \left(\frac{R+1}{R-1} \right) \right|^{-1}$

$\rightarrow \|x^* - x^{(k)}\|_A \leq \min_{\substack{\deg(q) \leq k \\ q(0)=1}} \left[\max_j |q(\lambda_j)| \right] \leq \left| T_k \left(\frac{R+1}{R-1} \right) \right|^{-1}$

The min of all q is smaller than any allowed q .
 $\rightarrow$ take normalized Chebyshev.
 $\rightarrow \max(T_k) = 1$.

WTS: $\left| T_n \left(\frac{R+1}{R-1} \right) \right| \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^n$

pf: $T_n(x)$, let $x = \frac{1}{2}(z + z^{-1})$ ← name of substitution, Znowski?

$$\rightarrow T_n(x) = \frac{1}{2}(z^n + z^{-n}) \Rightarrow T_n\left(\frac{x+x^{-1}}{2}\right) = \frac{x^n + x^{-n}}{2} \leftarrow \text{property of Chebyshev.}$$

$$\text{let } x = \frac{R+1}{R-1} = \frac{1}{2}(z + z^{-1}) \Rightarrow z = \frac{\sqrt{R}+1}{\sqrt{R}-1} \Rightarrow T_n\left(\frac{R+1}{R-1}\right) = \frac{1}{2}\left(\left(\frac{\sqrt{R}+1}{\sqrt{R}-1}\right)^n + \left(\frac{\sqrt{R}-1}{\sqrt{R}+1}\right)^n\right)$$

$$\rightarrow R = \frac{B}{\alpha} \geq 1 \Rightarrow \frac{\sqrt{R}+1}{\sqrt{R}-1} \geq 1 \rightarrow |T_n\left(\frac{R+1}{R-1}\right)|^{-1} = 2 \left(z^n + \cancel{z^{-n}}\right)^{-1} \quad (2) \geq 1$$

$$\leq 2 z^{-n} = 2 \left(\frac{\sqrt{R}-1}{\sqrt{R}+1}\right)^n$$

$$\therefore \boxed{\|x^* - x^{(n)}\|_A \leq \|x^* - x^{(0)}\|_A \cdot 2 \left(\frac{\sqrt{R}-1}{\sqrt{R}+1}\right)^n}$$

$$\frac{\|x^* - x^{(k)}\|_A}{\|x^* - x^{(0)}\|_A} \leq \inf_{q(\omega)=1} \max_{1 \leq j \leq k} q(\omega_j)$$

If you know what the eigenvalues look like, then the lemma can be useful.

If there are a lot of distinct eigenvalues, then you can't use Lagrange interpolation

→ slow

① If A is ill conditioned and has many eigenvalues → CG bad

② CG can be fast if $R \gg 1$ if eigenvalues are clustered.

③ The bound is on A -norm bound, not a 2-norm — maybe be bad.

pf: A SPD $\rightarrow Q^T A Q = \Lambda$, $c = Q^T v \rightarrow v = Q c$

v is a vector, and c is the vector in eigenspace.

$$\|v\|_2^2 = \sum_{j=1}^n c_j^2, \text{ but } \|v\|_A^2 := \langle v, Av \rangle = \sum_{j=1}^n c_j^2 \lambda_j$$

last time

very aggressive error bound → mostly only useful as a heuristic

$$\text{If } \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0, \|v\|_2^2 \leq \frac{1}{\lambda_n} \sum_{j=1}^n c_j^2 \lambda_j = \frac{\|v\|_A^2}{\lambda_n}$$

$$\rightarrow \lambda_n \|v\|_2^2 \leq \|v\|_A^2 \quad (\text{if } \lambda \sim 0 \text{ (A is almost singular), a small A-norm})$$

error is not the same as a small 2-norm error $\Rightarrow$ needs many more iterations.

error cics do can't cure about sparseness — it only makes implementations faster.

④ A is symmetric positive semidefinite, and $b \in \text{range}(A)$, then CG will just work.

→ we will do this on Wednesday. (super important in practice)

→ PDEs w/ Neumann B.C.)

⑤ If A isn't symmetric, you can make it SPD by multiplying by A^T

$$A^T A x = A^T b \quad \text{But this is spcr but because } \kappa(A^T A) = \kappa(A)^2 \quad \infty$$

The real remedy is GMRES. (do directly minimise the 2-norm (after we do preconditioning).)

PL4
We want $\frac{\sqrt{R}-1}{\sqrt{R}+1} \sim 1$. In general, if $A \approx I$, then the eigenvalues $\lambda_i \sim 1$.

→ the goal is to find $\tilde{A}$ such that $\tilde{A} A \approx I$ (or at least have more clustered λ_i).

We also want $\tilde{A}^{-1}$ to be cheap (ie fast to solve $\tilde{A} x = b$, not that we can store $\tilde{A}^{-1}$). Then $\tilde{A}^{-1} A b = \tilde{A}^{-1} b$ is a preconditioned system and $\tilde{A}$ is a preconditioner matrix.

We have a symmetry issue since $\tilde{A}^{-1} A$ is not always symmetric (even if $A/\tilde{A}$ are).

Remedy: let M SPD $\rightarrow M^{-1/2}$ exists and M^{-1} SPD $\rightarrow M^{-1} = U \Lambda U^T$, $\Lambda = \text{diag}(\lambda_i)$

$$\rightarrow M^{-1/2} = U \Lambda^{-1/2} U^T \quad y = M^{1/2} x \Rightarrow x = M^{-1/2} y$$

$$\rightarrow Ax = b \Rightarrow \underbrace{M^{-1/2} A M^{-1/2}}_{S''} y = \underbrace{M^{-1/2} b}_{\beta} = \beta$$

S'' SPD, if $M^{-1} \approx A^{-1}$, then this is easier to solve.

$Sy = \beta$ is easier to solve w/ CG.

• You don't want to build $S = M^{-1/2} A M^{-1/2}$, you want to show that CG on

$Sy = \beta$ is equal to solving $Ax = b$.

Thm: A is PSD, $\vec{b} \in R(A) \Rightarrow$ CG converges within n iterations.

$\hookrightarrow$ all $\lambda \geq 0$.

pf: A is PSD $\Rightarrow Q^T A Q = \Lambda = \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_r & & 0 \\ & & & \ddots & \\ 0 & & & & 0 \end{bmatrix}$

$R(A)$ = range of A = L.C. at column space
 $N(A)$ = nullspace of A = orthogonal to $R(A)$.

$r = \text{rank}(A)$, $Q^T Q = I$, $\lambda_i > 0$ are eigenvalues

decompose $\vec{v} \in \mathbb{R}^n$ into $R(A) \perp N(A) = R(A)^\perp$

$Q = [q_1, \dots, q_r, q_{r+1}, \dots, q_n] = [\underbrace{Q_1}_{\text{eigenvectors with } \lambda > 0}, \underbrace{Q_2}_{\text{eigenvectors with } \lambda = 0}]$

The first r rows span $R(A)$, the last $n-r$ rows span $N(A)$.

$\rightarrow \text{span}(Q_1) = R(A), \text{span}(Q_2) = N(A)$

$\forall \vec{v} \in \mathbb{R}^n, Q^T \vec{v} = \begin{bmatrix} Q_1^T \\ Q_2^T \end{bmatrix} \vec{v} = \begin{bmatrix} Q_1^T \vec{v} \\ Q_2^T \vec{v} \end{bmatrix} = \begin{bmatrix} \vec{v}_1' \\ \vec{v}_2' \end{bmatrix}$
 $\swarrow$ proj of $\vec{v}$ onto $R(A)$
 $\searrow$ proj of $\vec{v}$ onto $N(A)$.

$\rightarrow Q \begin{bmatrix} \vec{v}_1' \\ \vec{v}_2' \end{bmatrix} = Q Q^T \vec{v} = \vec{v}$

w/ CG, $\vec{r} = \vec{b} - A\vec{x} \Rightarrow Q^T \vec{r} = Q^T \vec{b} - Q^T A \vec{x} = \Lambda Q^T \vec{x}$
 $\begin{bmatrix} \vec{r}_1' \\ \vec{r}_2' \end{bmatrix} = \begin{bmatrix} \vec{b}_1' \\ \vec{b}_2' \end{bmatrix} - \begin{bmatrix} \Lambda_r & 0 \end{bmatrix} \begin{bmatrix} \vec{x}_1' \\ \vec{x}_2' \end{bmatrix}$

$\Rightarrow \vec{r}_1' = \vec{b}_1' - \Lambda_r \vec{x}_1', \vec{r}_2' = \vec{b}_2'$

$\langle \vec{r}, \vec{r} \rangle = \langle \vec{r}_1', \vec{r}_1' \rangle + \langle \vec{r}_2', \vec{r}_2' \rangle = \langle \vec{r}_1', \vec{r}_1' \rangle + \langle \vec{b}_2', \vec{b}_2' \rangle$

$\langle A\vec{p}, \vec{p} \rangle = \langle \vec{p}', \Lambda_r \vec{p}' \rangle + \langle \vec{p}_2', 0 \cdot \vec{p}_2' \rangle = \|\vec{p}'\|_A$

$Q^T A = Q^T A Q Q^T = \Lambda Q^T$

$\vec{r}_0 = \vec{b} - A\vec{x}$
 $\vec{p}_0 = \vec{r}_0$
for $i = 1, 2, \dots$:
 $\alpha_i = \frac{\langle \vec{r}_i, \vec{r}_i \rangle}{\langle A\vec{p}_i, \vec{p}_i \rangle}$
 $\vec{x}_{i+1} = \vec{x}_i + \alpha_i \vec{p}_i$
 $\vec{r}_{i+1} = \vec{r}_i - \alpha_i A\vec{p}_i$
 $\beta_i = \frac{\langle \vec{r}_{i+1}, \vec{r}_{i+1} \rangle}{\langle \vec{r}_i, \vec{r}_i \rangle}$
 $\vec{p}_{i+1} = \vec{r}_{i+1} + \beta_i \vec{p}_i$

Algorithm in $R(A), N(A)$

$R(A)$

$N(A)$

IF $\vec{b} \in R(A), \vec{b}_2' = 0$

$\vec{r}_0' = \vec{b}_1' - \Lambda_r \vec{x}_0'$
 $\vec{p}_0' = \vec{r}_0'$
 $\alpha_i = \frac{\langle \vec{r}_i', \vec{r}_i' \rangle + \langle \vec{b}_2', \vec{b}_2' \rangle}{\langle \vec{p}_i', \vec{p}_i' \rangle}$

$0 = \vec{r}_0^2 = \vec{b}_2^2 \leftarrow$ same for all $\vec{r}_i$, it never changes in $N(A)$
 $0 = \vec{p}_0^2 = \vec{b}_2^2$

$\vec{x}_{i+1}' = \vec{x}_i' + \alpha_i \vec{p}_i'$

$0 = \vec{r}_{i+1}^2 = \vec{b}_2^2$

$\vec{p}_{i+1}^2 = \vec{b}_2^2 + \beta_i \vec{p}_i^2 \rightarrow 0$, induction

$\beta_i = \frac{\langle \vec{r}_{i+1}', \vec{r}_{i+1}' \rangle + \langle \vec{b}_2', \vec{b}_2' \rangle}{\langle \vec{r}_i', \vec{r}_i' \rangle + \langle \vec{b}_2', \vec{b}_2' \rangle}$
 $\vec{p}_{i+1}' = \vec{r}_{i+1}' + \beta_i \vec{p}_i'$

- CG leaves the nullspace component of vectors the same or solves for $\vec{b}' \in R(A)$
- we are solving it over a smaller space, $r = \text{rank}(A)$.
- $\rightarrow$ CG converges in r steps.

A singular, $\vec{b} \in \mathbb{R}(A) \sim$ consistent $\Rightarrow$ infinitely many solutions to $A\vec{x} = \vec{b}$.

$\Rightarrow$ converge to Least-Square solution.

- CG can exploit sparseness; pseudo-inverse cannot.

Eigenvalues:

QR: $A_i = Q_i R_i \Rightarrow A_{i+1} = R_i Q_i, R_i \rightarrow \Lambda$

Power: $A_{i+1} = A_i^2$.

$$A\vec{x} = \vec{b} \text{ is hard} \Rightarrow \tilde{A}^{-1} A \vec{b} = \tilde{A}^{-1} \vec{b}$$

A, M are SPD $\Rightarrow M^{-1/2}$ exists, $M^{-1} = U \Lambda U^T, M^{-1/2} = U \Lambda^{1/2} U^T$

$$A\vec{x} = \vec{b} \Rightarrow \underbrace{M^{-1/2} A M^{-1/2}}_{\text{still SPD}} \vec{y} = M^{-1/2} \vec{b}, \quad \vec{x} = M^{-1/2} \vec{y}$$

$\hookrightarrow$ PCH $\Rightarrow$ plug into CG

simplified version ($M^{-1/2}$ disappears)

$k=0, \vec{r}_0 = \vec{b} - A\vec{x}, M\vec{z}_0 = \vec{r}_0$ preconditioned residual.

while $\|\vec{r}_k\| > \epsilon \|\vec{b}\|$ {

$k = k+1$

if $k=1: \vec{p}_k = \vec{z}_0$

if $k>1:$

$$\tau = \frac{\vec{r}_{k-1}^T \vec{z}_{k-1}}{\vec{r}_{k-1}^T \vec{z}_{k-1}}$$

$$\vec{p}_k = \vec{z}_{k-1} + \tau \vec{p}_{k-1}$$

$$\mu = \frac{\vec{r}_{k-1}^T \vec{z}_{k-1}}{\vec{p}_{k-1}^T \vec{p}_{k-1}}$$

$$\vec{x}_k = \vec{x}_{k-1} - \mu \vec{p}_k$$

$$\vec{r}_k = \vec{r}_{k-1} - \mu A \vec{p}_k, M \vec{z}_k = \vec{r}_k$$

}

return $\vec{x}_k$.

Prmk: PCH only needs to solve $M\vec{z} = \vec{r}$

$\rightarrow$ need to solve $M\vec{z} = \vec{r}$ fast

$\rightarrow$ we also want $K(M^{-1} A M^{-1}) \sim I$.

$\rightarrow$ motivates choice of preconditioner matrix

$\downarrow$

1) M is SPD

2) $M \sim A$

3) $M\vec{z} = \vec{r}$ is easy

$\rightarrow$ stationary splitting are easy to solve

$$G3: D \vec{x}^{(n+1)} = -(L+U) \vec{x}^{(n)} \text{ motivates } M=D$$

- If we want to do multiple iterations, we can use

$$D \vec{x}^{(1)} = -(L+U) \vec{x}^{(0)}, D \vec{x}^{(2)} = -(L+U) \vec{x}^{(1)}$$

$$D \vec{x}^{(4)} = (L+U) D^{-1} (L+U) \vec{x}^{(0)}$$

- $M\vec{z} = \vec{r}$ is done w/ p iterations at G3, G5 or SSOR/better.

- trivial to do, but doesn't help much

- in practice G3 preconditioned CG is called "cg" bc it is a really minimal effort speed up.

$$- DCN = \text{diag}(A) + \epsilon I$$

Block GS preconditioner

$$A = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1n} \\ A_{21} & \ddots & & \vdots \\ \vdots & & & \\ A_{n1} & \dots & A_{nn} \end{bmatrix}, \quad M = \begin{bmatrix} A_{11} & & & \\ & A_{22} & & \\ & & \ddots & \\ & & & A_{nn} \end{bmatrix}$$

- usually, these are 2×2 or 5×5 blocks because there are cheap closed formulas for inverse.
- M^{-1} is the same as doing $A_{ii} z_i = r_i$ (also easy to parallelize).
- Since it's block wise, it's better than pure diagonal D .
- What about a banded diagonal system?

Incomplete Matrix Factorization

$$A = L L^T \leftarrow \text{cholesky} \Rightarrow L \text{ can be dense}$$

incomplete cholesky $A \approx L L^T$ so L has the same sparsity pattern as A
 $\leftarrow$ lower part of A is the same as upper bc SPD.

- can compute by zeroing out things during Gaussian Elimination). $\leftarrow O(n^4)$ instead of normal $O(n^3)$ bc sparse
- $L L^T \approx A$ is sparse, and fast to solve ($L w = r, L^T z = w$)
 - $O(n)$ instead of $O(n^2)$ w/ normal cholesky.

Current Research: Designing Preconditioners on insight on where the system came from.

$$\text{ex: } -\nabla \cdot (a(x, y) \nabla u) = f(x, y), \quad \Omega = [0, 1]^2, \quad u(\partial\Omega) = 0.$$

- w/ finite difference, the discretized operator $Au = f$ is still a linear operator.
- A is location dependent (not constant / symmetric / Laplacian).
- Krylov solver is better than CG here.

- The most expensive calculation is applying it to a vector to get AP_k .
 $\leftarrow$ simple SPD discrete Laplacian
- Still need a preconditioner: use $a(x, y) = c \Rightarrow M = c D_h$ (much simpler)
- $Mz = r$ can be solved "quickly" w/ PCG.

- You shouldn't store A or D because it's BICG, all we need is their action (w/ a function).

$$\hookrightarrow D \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \frac{-2u_2 + u_1 + u_3}{h^2} \quad \leftarrow \text{implement this as a function instead of a matrix.} \quad f \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \frac{u_1 - 2u_2 + u_3}{h^2}$$

Domain Decomposition Preconditioner

- Ω at a PDE has subregions that are disjoint / slightly overlapping: $\Omega = \bigcup_i \Omega_i$
- Preconditioner matrix is solving Ω_i by pretending they are not overlapping (average at interface).
- implement this 😊
- CFD for blood flow in ppl is a good example at distinct subregions

Multigrid Preconditioner



- Divide and conquer: instead of solving $n \times n$ $Ax=b$, we discretize your problem into a coarser grid ($\frac{n}{2} \times \frac{n}{2}, \frac{n}{4} \times \frac{n}{4}, \dots$) and solve this simpler system.
 - ① Transfer r from normal to coarse grid
 - ② solve $Mz=r$ on coarse grid
 - ③ Interpolate z onto original grid (linear in 1D, bilinear in 2D, ...)
- these are called smoothers
- Very good at removing low frequency residual errors.
 - Do 3 iters at G_5 or G_5 before and after ① ② ③ ("V" cycle)
 - There are a lot of ways to do this "V" cycle.

Neural Network Preconditioner

- train deep NN_{Θ} s.t. $z = NN_{\Theta}(r)$ is close to $A^{-1}r$
- sample lots at r_s and use slower methods to generate $A^{-1}r$ data $\Rightarrow$ train $\Rightarrow NN \sim M$.

$$\bar{x} = \sum_{i=1}^n \alpha_i p_i, \quad \hat{x} = \sum_{i=1}^k \delta_i p_i$$

want $\|\bar{x} - \hat{x}\|_A$ minimized at $\delta_i \neq \alpha_i$

$$\bar{x} - \hat{x} = \sum_{i=1}^k (\alpha_i - \delta_i) p_i + \sum_{i=k+1}^n \alpha_i p_i, \quad \text{Let } z_i = \begin{cases} \alpha_i - \delta_i & \text{if } i \leq k \\ \alpha_i & \text{if } i > k \end{cases}$$

$$\rightarrow \bar{x} - \hat{x} = \sum_{i=1}^n z_i p_i$$

$$\rightarrow \langle \bar{x} - \hat{x}, A(\bar{x} - \hat{x}) \rangle = \left\langle \sum_{i=1}^n z_i p_i, A \sum_{i=1}^n z_i p_i \right\rangle$$

From the fact that $i \neq j$, $\langle p_i, A p_j \rangle = 0$ by the definition of A -orthogonality.

$$\Rightarrow \sum_{i=1}^n \langle z_i p_i, A z_i p_i \rangle$$

$$= \sum_{i=1}^n z_i^2 \|p_i\|_A^2$$

$$\rightarrow \|\bar{x} - \hat{x}\|_A = \sqrt{\sum_{i=1}^n z_i^2 \|p_i\|_A^2}$$

Note that if $i > k$, $z_i = \alpha_i$, which we have no control over.

However, if $i \leq k$, $z_i = (\alpha_i - \delta_i)^2$ is minimized at $z_i = 0$

$$\rightarrow 0 = \alpha_i - \delta_i \Rightarrow \alpha_i = \delta_i$$

We'll show $\text{span}\{p_1, p_2, \dots, p_k\} = \text{span}\{r_0, \dots, r_{k-1}\}$ w/ induction

For the base case, we have $\text{span}\{r_0\} = \text{span}\{p_1\}$ since we initially set $p_0 = r_0$.

For the I.H., assume $\text{span}\{p_0, \dots, p_i\} = \text{span}\{r_0, \dots, r_i\}$. WTS $\text{span}\{p_0, \dots, p_{i+1}\} = \text{span}\{r_0, \dots, r_{i+1}\}$

It suffices to show that $p_{i+1} \in \text{span}\{r_0, \dots, r_i\}$, since every other element in $\{p_0, \dots, p_{i+1}\}$ is in $\text{span}\{r_0, \dots, r_{i+1}\} \in \text{span}\{r_0, \dots, r_{i-1}, r_i\}$ by our I.H. we set this.

from Fact 4 in Problem 1: $p_{i+1} = r_i + \beta_i p_i$ since $p_i \in \text{span}\{r_0, \dots, r_i\}$ by I.H. as well.

Thus, we move onto $\text{span}\{r_0, \dots, r_{k-1}\} = \text{span}\{b, Ab, \dots, A^{k-1}b\}$. To arrive at an identical inductive argument as last time, we need to show that a linear combination in

$\text{span}\{r_0, \dots, r_{k-1}\}$ is equal to $A^{k-1}b$ (the base case is once again trivial since $r_0 = b$).

For this, our I.H. involves assuming $A^{k-2}b$ is a linear combination of $\{r_0, \dots, r_{k-1}\}$.

By the previous statement, this is also a linear combination of $\{p_1, \dots, p_k\}$. Thus, $p_k = \tilde{p} + A^{k-2}b$

for a reachable $\tilde{p}$. From Fact 5, we now get $r_k = r_{k-1} - \alpha_k A \cdot (\tilde{p} + A^{k-2}b)$

$$\rightarrow A^{k-1}b = \frac{r_{k-1} - r_k}{\alpha_k} = A \tilde{p} \in \text{span}\{r_0, \dots, r_{k-1}\}$$